PEMROGRAMAN BERORIENTASI OBJEK 2026

Relasi Antarkelas

Memakai, mengenal, atau memiliki: cara objek bekerja sama, dan program mulai terdiri atas banyak *file*

PERTEMUAN

05

dari 16

Program Studi Informatika · Universitas Mulawarman
Jalur praktik: Python dan Java

Tujuan Pembelajaran

Setelah pertemuan ini, kamu mampu:

1. Menjelaskan mengapa objek perlu saling berhubungan dan saling **mengirim pesan**.
2. Membedakan **dependensi**, **asosiasi**, **agregasi**, dan **komposisi** dengan analogi sehari-hari.
3. Membaca dan menentukan **multiplisitas** sebuah hubungan: `1`, `0..1`, `*`, dan `1..*`.
4. Menerjemahkan setiap jenis hubungan ke dalam kode: parameter, atribut berisi objek, *list* berisi objek, dan objek yang dibuat di dalam objek lain.
5. Melindungi *list* di dalam objek dengan mengembalikan **salinan**, serta menangani pencarian yang tidak menemukan apa-apa.
6. Menyusun program menjadi beberapa *file* dan menjalankannya dari folder yang benar.

Mengingat Kembali



Dari pertemuan sebelumnya

- **Enkapsulasi** menyembunyikan data di dalam kelas; pihak luar memakai pintu resmi seperti *getter*, *setter*, dan *property*.
- Data yang tidak masuk akal ditolak sejak di pintu masuk dengan melempar *exception*.
- Prinsip ***tell, don't ask***: suruh objek mengerjakan tugasnya sendiri.
- Variabel hanya menyimpan **referensi** ke objek (Pertemuan 03). Referensi inilah "tali" penghubung antarobjek.

Agenda Hari Ini

1. Objek tidak bisa bekerja sendirian
2. Empat jenis hubungan antarobjek
3. Agregasi dan komposisi
4. Multiplisitas, salinan *list*, dan pencarian
5. Program banyak *file*
6. Studi kasus KedaiKita

BAGIAN 1

1

Objek Tidak Bisa Bekerja Sendirian

Program yang baik terdiri atas banyak kelas kecil yang bekerja sama.

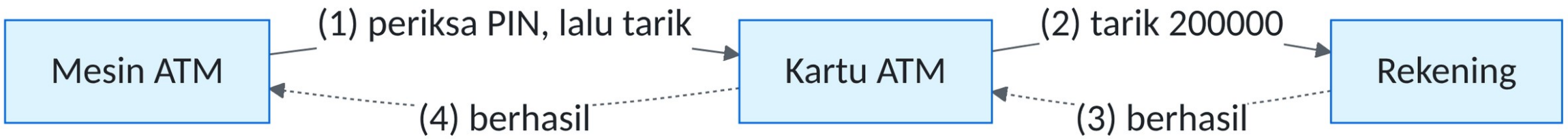
Budi Memesan Apa?

Relasi antarkelas

Hubungan yang membuat objek dari satu kelas bisa **memakai, mengenal, atau memiliki** objek dari kelas lain sehingga objek-objek itu bisa bekerja sama.

- Objek Budi dan objek kopi hidup terpisah, jadi tidak ada yang mencatat bahwa Budi memesan 2 kopi.
- Satu kelas raksasa yang mengerjakan semuanya sulit dibaca, sulit diuji, dan sulit diubah.
- Syarat objek A bisa mengirim pesan ke B: A **memegang referensi** ke B, lewat parameter atau atribut.

Objek Saling Mengirim Pesan



Panah utuh adalah pesan yang dikirim, panah putus-putus adalah jawaban yang kembali. Angka menunjukkan urutannya.

BAGIAN 2

2

Empat Jenis Hubungan Antarobjek

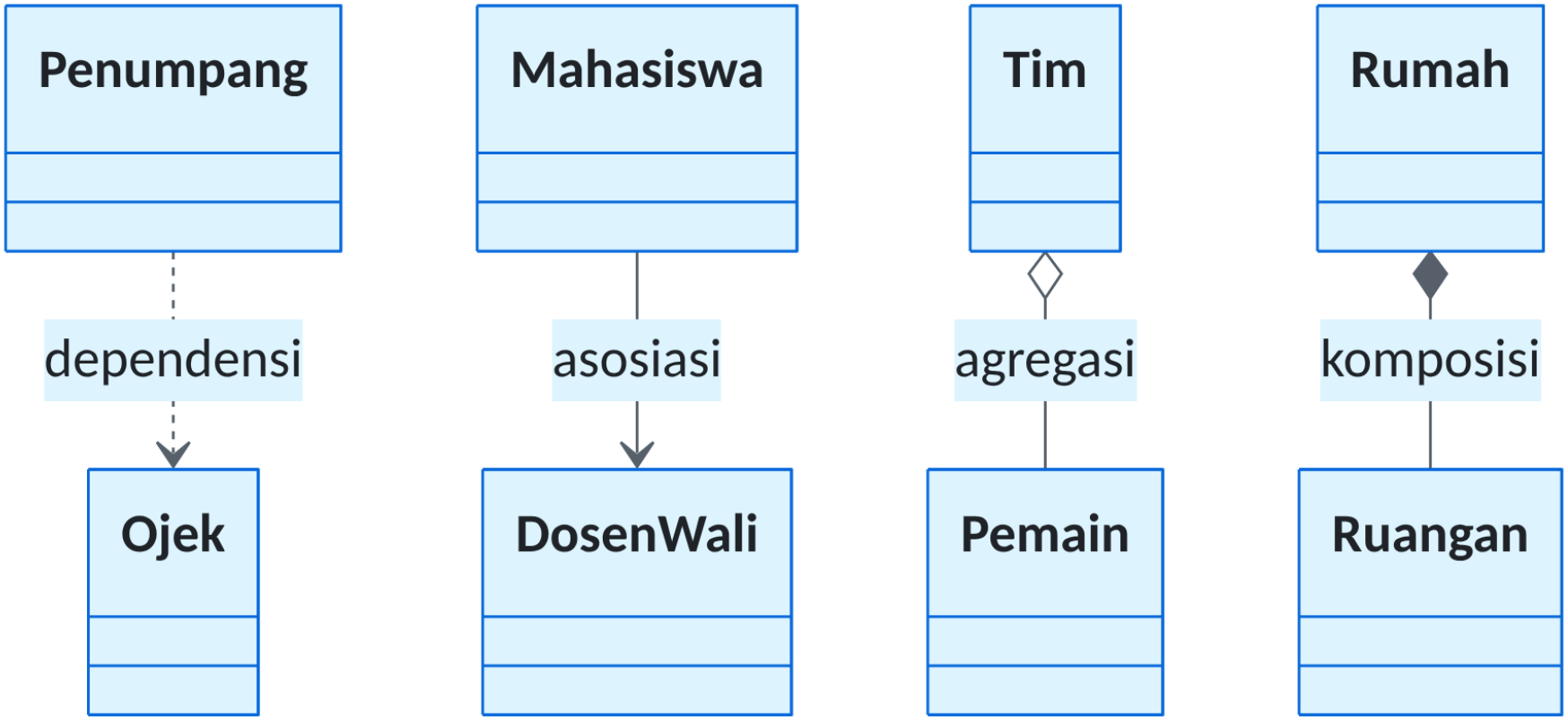
Dari yang paling longgar sampai yang paling erat.

Empat Jenis Hubungan

Jenis relasi	Arti sederhana	Contoh sehari-hari	Contoh di KedaiKita
Dependensi (*dependency*)	memakai sebentar	mahasiswa memesan ojek	`Kedai` memakai `Pelanggan` saat membuat pesanan
Asosiasi (*association*)	mengenal	mahasiswa dan dosen walinya	`Pesanan` mengenal `Pelanggan` pemiliknya
Agregasi (*aggregation*)	punya, tetapi bagiannya bisa hidup sendiri	tim futsal dan pemainnya	`Kedai` punya daftar `Menu`
Komposisi (*composition*)	punya, dan bagiannya ikut hilang	rumah dan kamarnya	`Pesanan` punya `ItemPesanan`

Urutan dari yang paling longgar ke yang paling erat: dependensi, asosiasi, agregasi, komposisi.

Setiap Relasi Punya Bentuk Garis



Putus-putus berpanah: dependensi. Utuh berpanah: asosiasi. Belah ketupat kosong atau penuh di sisi pemilik: agregasi atau komposisi.

Dependensi: Memakai Sebentar

Analogi

- Ayu memesan ojek daring dari kos ke kampus. Selama perjalanan, Ayu **memakai** jasa pengemudi.
- Begitu sampai, urusannya selesai. Ayu tidak “menyimpan” pengemudi itu.
- Besok Ayu bisa saja diantar **pengemudi lain**.

Dunia ojek	Di kode
Ayu memakai pengemudi selama perjalanan	B datang sebagai **parameter** *method* milik A
Urusan selesai begitu sampai	B **tidak disimpan** sebagai atribut A
Besok bisa diantar pengemudi lain	Setiap pemanggilan boleh memakai objek yang berbeda

Asosiasi: Atribut Berisi Objek Lain

Mahasiswa menyimpan **referensi** dosen walinya di atribut. Ayu dan Bima mengenal objek dosen yang sama.

Python python/contoh/contoh02_asosiasi.py

```python
class Mahasiswa:
    """Mahasiswa yang mengenal dosen walinya."""

    def __init__(self, nama: str):
        self._nama = nama
        self._dosen_wali: DosenWali | None = None

    def atur_dosen_wali(self, dosen: DosenWali):
        self._dosen_wali = dosen
# ... di dalam main()
ayu.atur_dosen_wali(bu_rina)
bima.atur_dosen_wali(bu_rina)
# ... (dilewati)
bu_rina.pindah_ruangan("D-105")
```

Java java/contoh/Contoh02Asosiasi.java

```java
class Mahasiswa {
    private String nama;
    private DosenWali dosenWali;
    // ... (dilewati)
    public void aturDosenWali(DosenWali dosen) {
        this.dosenWali = dosen;
    }
    // ... (dilewati)
}
// ... di dalam main
ayu.aturDosenWali(buRina);
bima.aturDosenWali(buRina);
// ... (dilewati)
buRina.pindahRuangan("D-105");
```

BAGIAN 3

3

Agregasi dan Komposisi

Sama-sama hubungan punya, tetapi siklus hidup bagiannya berbeda.

Agregasi: *List* Berisi Objek dari Luar

Pemain dibuat **di luar** tim, lalu diserahkan lewat `rekrut()`. Bayu boleh ikut dua tim sekaligus.

Python python/contoh/contoh03_agregasi.py

```python
class Tim:
    # ... (dilewati)
    def __init__(self, nama: str):
        self._nama = nama
        self._daftar_pemain: list[Pemain] = []

    def rekrut(self, pemain: Pemain):
        self._daftar_pemain.append(pemain)
        # ... (dilewati)
# ... di dalam main()
tim_kelas.rekrut(andi)
tim_kelas.rekrut(bayu)
tim_kelas.rekrut(candra)
tim_fakultas.rekrut(bayu)
```

Java java/contoh/Contoh03Agregasi.java

```java
class Tim {
    private String nama;
    private List<Pemain> daftarPemain;
    public Tim(String nama) {
        this.nama = nama;
        this.daftarPemain = new ArrayList<>();
    }
    public void rekrut(Pemain pemain) {
        daftarPemain.add(pemain);
        // ... (dilewati)
    }
    // ... (dilewati)
}
// ... di dalam main
timKelas.rekrut(andi);
timKelas.rekrut(bayu);
timKelas.rekrut(candra);
timFakultas.rekrut(bayu);
```

Komposisi: Bagian Dibuat di dalam Pemiliknya

Di dalam `bangun_ruangan()`, rumah membuat sendiri objek `Ruangan`. Kode luar hanya mengirim **data**.

Python python/contoh/contoh04_komposisi.py

```python
def bangun_ruangan(self, nama: str, panjang: int, lebar: int):
    ruangan = Ruangan(nama, panjang, lebar)
    self._daftar_ruangan.append(ruangan)
# ... (dilewati)
rumah = Rumah("Pak Hasan")
rumah.bangun_ruangan("Ruang tamu", 4, 3)
rumah.bangun_ruangan("Kamar tidur", 3, 3)
rumah.bangun_ruangan("Dapur", 3, 2)
```

Java java/contoh/Contoh04Komposisi.java

```java
public void bangunRuangan(String nama, int panjang, int lebar) {
    Ruangan ruangan = new Ruangan(nama, panjang, lebar);
    daftarRuangan.add(ruangan);
}
// ... (dilewati)
Rumah rumah = new Rumah("Pak Hasan");
rumah.bangunRuangan("Ruang tamu", 4, 3);
rumah.bangunRuangan("Kamar tidur", 3, 3);
rumah.bangunRuangan("Dapur", 3, 2);
```

Agregasi atau Komposisi?

Pertanyaan	Agregasi	Komposisi
Bisakah bagian tetap ada tanpa keseluruhannya?	bisa (pemain tanpa tim)	tidak (kamar tanpa rumah)
Bisakah satu bagian dimiliki dua keseluruhan sekaligus?	bisa (pemain di dua tim)	tidak (item hanya milik satu pesanan)
Siapa yang biasanya membuat objek bagian?	pihak luar	keseluruhan itu sendiri

Masih ragu? Pikirkan siklus hidup: kalau pemiliknya hilang, apakah bagian ini masih masuk akal untuk ada?

BAGIAN 4

4

Multiplisitas, Salinan *List*, dan Pencarian

Berapa banyak objek yang terhubung, dan cara menjaga datanya.

Multiplisitas: Berapa Banyak?

Notasi	Dibaca	Contoh	Bentuk di kode
`1`	tepat satu	satu pesanan dimiliki tepat satu pelanggan	atribut **wajib**, diisi lewat konstruktor
`0..1`	nol atau satu	mahasiswa baru boleh belum punya dosen wali	atribut boleh kosong (`None`/`null`)
`*`	nol atau lebih	tim baru boleh belum punya pemain	*list* yang boleh kosong
`1..*`	minimal satu	grup belajar minimal punya satu anggota, yaitu ketuanya	*list* yang sejak awal berisi minimal satu objek

Cara membaca: ambil **satu** objek di satu sisi, lalu lihat angka di **ujung seberang**.

Salinan *List* dan Pencarian yang Gagal

Jangan serahkan *list* aslinya

- *Getter* yang menyerahkan *list* asli membuat isinya bisa ditambah **tanpa lewat** *method* penjaga aturan.
- Serahkan **salinan**: `list(daftar)` di Python, `new ArrayList<>(daftar)` di Java.
- Seperti menyerahkan fotokopi KTP, bukan KTP aslinya.
- Yang disalin hanya *list*-nya; objek di dalamnya tetap objek yang sama.

Pencarian tanpa hasil

- `cari_menu("M99")` mengembalikan `None` (Python) atau `null` (Java).
- Pemanggil **wajib memeriksa** hasilnya: Python `if menu is None:`, Java `if (menu == null)`.
- Jika langsung dipakai, program berhenti dengan `AttributeError` atau `NullPointerException`.

BAGIAN 5

5

Program Banyak *File*

Satu kelas, satu *file*, dan satu pintu masuk.

Program Banyak *File* di Python dan Java

Hal	Python	Java
Satu *file* disebut	modul (*module*)	*file* sumber
Aturan nama *file*	bebas, lazimnya huruf kecil: `pesanan.py`	**wajib** sama dengan nama kelas `public`-nya: `Pesanan.java`
Isi satu *file*	boleh beberapa kelas: `ItemPesanan` dan `Pesanan`	satu kelas `public` per *file*
Memakai kelas dari *file* lain	wajib `import`: `from menu import Menu`	tanpa `import` jika satu folder
Menjalankan	`python main.py`	`javac *.java`, lalu `java Main`

Masuk dulu ke folder `studi-kasus` sebelum mengetik perintah di terminal. Yang dijalankan selalu pintu masuknya: `main.py` atau `Main`.

STUDI KASUS



Studi Kasus KedaiKita

Akhirnya KedaiKita bisa mencatat **siapa memesan apa**.

Tiga Kelas Baru di KedaiKita

Studi kasus KedaiKita

Kelas baru	Tugasnya
`ItemPesanan`	satu baris pesanan: menu apa dan berapa porsi, beserta subtotalnya
`Pesanan`	pesanan seorang pelanggan: bernomor otomatis, berisi item, mencetak struk
`Kedai`	“pusat” kafe: menyimpan dan mencari menu, membuat dan mencatat pesanan

`Menu` dan `Pelanggan` hampir tidak berubah, tetapi penghitung `jumlah_menu` dihapus; kini cukup `kedai.jumlah_menu()`.

Rancangan Relasi KedaiKita

Studi kasus KedaiKita

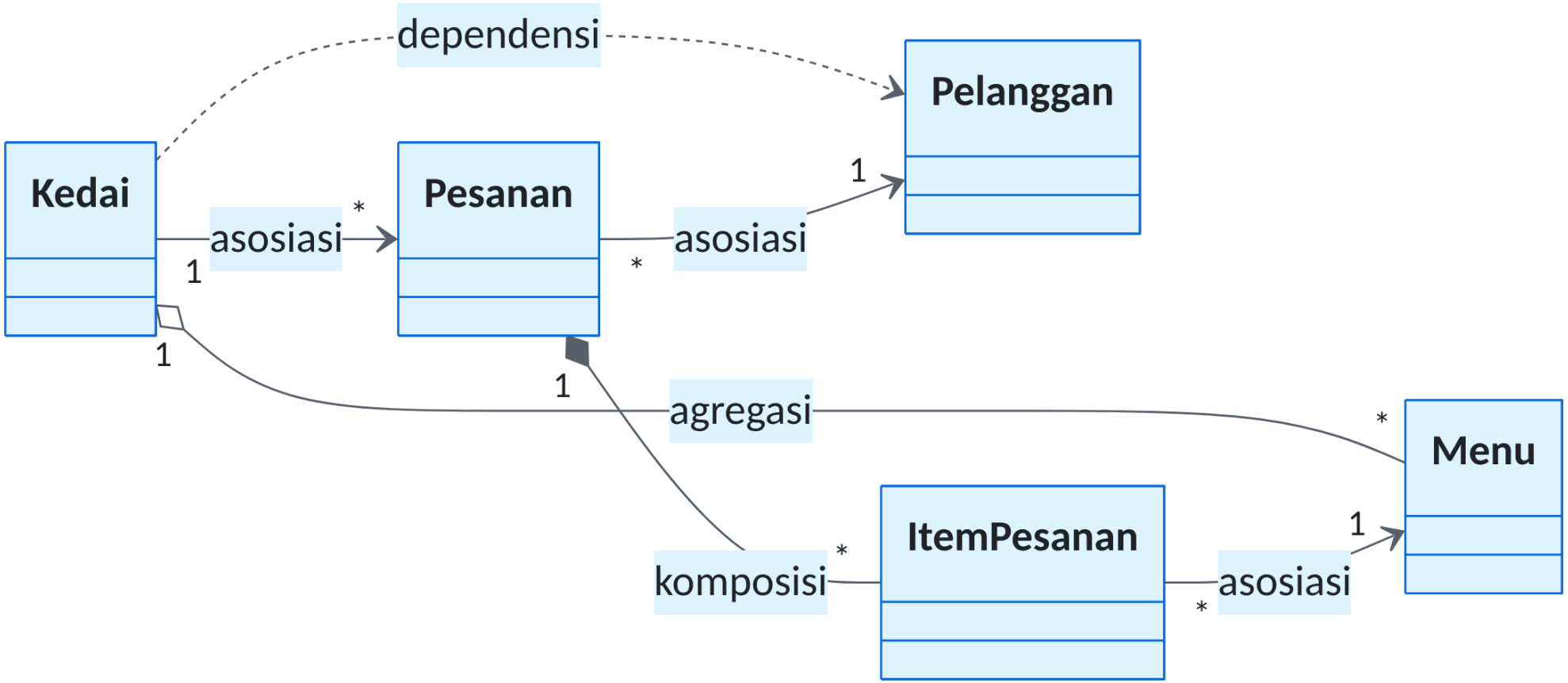


Diagram diringkas: hanya kelas dan garis relasinya. Diagram kelas lengkapnya kita buat di Pertemuan 06.

Pesanan Membuat Item dan Menyuruh Menu

Studi kasus KedaiKita

`tambah_item()` membuat objek item sendiri (komposisi), lalu menyuruh menu mengurangi stoknya.

Python python/studi-kasus/pesanan.py

```python
def tambah_item(self, menu: Menu, jumlah: int):
    item = ItemPesanan(menu, jumlah)
    menu.kurangi_stok(jumlah)
    self._daftar_item.append(item)

def hitung_total(self) -> int:
    total = 0
    for item in self._daftar_item:
        total += item.hitung_subtotal()
    return total
```

Java java/studi-kasus/Pesanan.java

```java
public void tambahItem(Menu menu, int jumlah) {
    ItemPesanan item = new ItemPesanan(menu, jumlah);
    menu.kurangiStok(jumlah);
    daftarItem.add(item);
}

public int hitungTotal() {
    int total = 0;
    for (ItemPesanan item : daftarItem) {
        total += item.hitungSubtotal();
    }
    return total;
}
```

Miskonsepsi yang Sering Terjadi

Anggapan keliru	Yang benar
“Agregasi dan komposisi itu sama saja karena sama-sama punya.”	Bedanya pada siklus hidup dan kepemilikan. Bagian agregasi bisa hidup sendiri; bagian komposisi dibuat pemiliknya dan ikut hilang bersamanya.
“Asosiasi berarti objek B disalin ke dalam objek A.”	Yang disimpan hanyalah **referensi**. Objek B tetap satu; jika B berubah, semua objek yang mengenalnya melihat perubahan itu.
“Hubungan antarobjek harus selalu dua arah.”	Hubungan satu arah sering kali cukup dan lebih mudah dijaga. Hubungan dua arah menuntut kedua sisi selalu diperbarui bersamaan.

Kuis Cepat

Manakah pasangan yang paling tepat digolongkan sebagai **komposisi**?

A. Tim futsal dan para pemainnya

B. *Playlist* dan lagu-lagu di dalamnya

C. Mahasiswa dan dosen walinya

D. Pesanan dan baris-baris item di dalamnya

Kuis Cepat: Jawaban

Manakah pasangan yang paling tepat digolongkan sebagai **komposisi**?

A Tim futsal dan para pemainnya

B *Playlist* dan lagu-lagu di dalamnya

C Mahasiswa dan dosen walinya

✓ Pesanan dan baris-baris item di dalamnya

Jawaban: D

Baris item seperti “2 x Kopi Susu Gula Aren” dibuat oleh pesanannya, hanya milik satu pesanan, dan tidak bermakna tanpanya: komposisi. Pemain (A) dan lagu (B) bisa hidup sendiri sehingga termasuk agregasi. Mahasiswa dan dosen wali (C) saling mengenal tanpa hubungan keseluruhan dan bagian, jadi asosiasi.

Kuis Cepat

Sebuah diagram menuliskan multiplisitas `0..1` di ujung `DosenWali` pada hubungan `Mahasiswa --> DosenWali`. Artinya ...

A Seorang mahasiswa boleh belum punya dosen wali, dan paling banyak punya satu

B Setiap mahasiswa wajib punya tepat satu dosen wali

C Seorang mahasiswa boleh punya banyak dosen wali

D Seorang dosen wali boleh tidak membimbing mahasiswa sama sekali

Kuis Cepat: Jawaban

Sebuah diagram menuliskan multiplisitas `0..1` di ujung `DosenWali` pada hubungan `Mahasiswa --> DosenWali`. Artinya ...

✓ Seorang mahasiswa boleh belum punya dosen wali, dan paling banyak punya satu

B Setiap mahasiswa wajib punya tepat satu dosen wali

C Seorang mahasiswa boleh punya banyak dosen wali

D Seorang dosen wali boleh tidak membimbing mahasiswa sama sekali

Jawaban: A

`0..1` berarti nol atau satu, dibaca dari sudut pandang **satu** mahasiswa karena ditulis di ujung `DosenWali`. Di kode, wujudnya atribut yang awalnya `None`/`null`. B adalah arti `1`, C adalah arti `*`, dan D membicarakan sudut pandang dosen, yang ditentukan angka di ujung `Mahasiswa`.

Ringkasan

- Program yang baik terdiri atas banyak kelas kecil yang **saling mengirim pesan**; syaratnya, pengirim memegang **referensi**.
- **Dependensi**: memakai sebentar; objek lain hanya datang sebagai parameter atau variabel lokal.
- **Asosiasi**: mengenal; objek lain disimpan sebagai atribut. Hubungan punya arah, dan satu arah sering kali cukup.
- **Agregasi**: *list* berisi objek yang dibuat di luar. **Komposisi**: objek bagian dibuat di dalam pemiliknya.
- **Multiplisitas** (`1`, `0..1`, `*`, `1..*`) menyatakan berapa banyak objek yang terhubung, dan kode harus menjaganya.
- *Getter* mengembalikan **salinan** *list*; hasil pencarian `None`/`null` wajib diperiksa sebelum dipakai.
- Program besar dipecah menjadi beberapa *file*; yang dijalankan selalu pintu masuknya, dari folder `studi-kasus`.

Langkah Berikutnya

- Jalankan contoh 01—07 (di Python juga contoh 08) dan studi kasus KedaiKita dari folder yang benar.
- Kerjakan latihan, lalu cek pemahamanmu dengan kuis 10 soal.

Buka bahan pertemuan ini di GitHub:

- Konsep
- Praktik Python
- Praktik Java
- Jembatan Python ↔ Java
- Latihan
- Kuis 10 soal

Pindai untuk membuka folder pertemuan ini di GitHub

github.com/antonprafanto/PBO2026/tree/master/Pertemuan-05-Relasi-Antarkelas